DSI STUDENT PORTAL

Exhaustive Low-Level Technical Architecture & Engineering Specification

Target Audience:	AI Technical Interviewer / Staff-Level System Design Panel
Candidate Benchmark:	10+ LPA Software Development Engineer (SDE-1 / SDE-2)
Target System:	Decoupled Vanilla JS SPA + Node.js/Express 5.1 + MongoDB (Mongoose 8) + Cloudinary + Gemini 1.5
Core Focus Areas:	JWT RBAC state machines, buffer-to-cloud streams, in-memory PDF parsing, scale limits at 10k QPS, and defensive security
Date & Version:	October 2026 | System Version 1.0.0

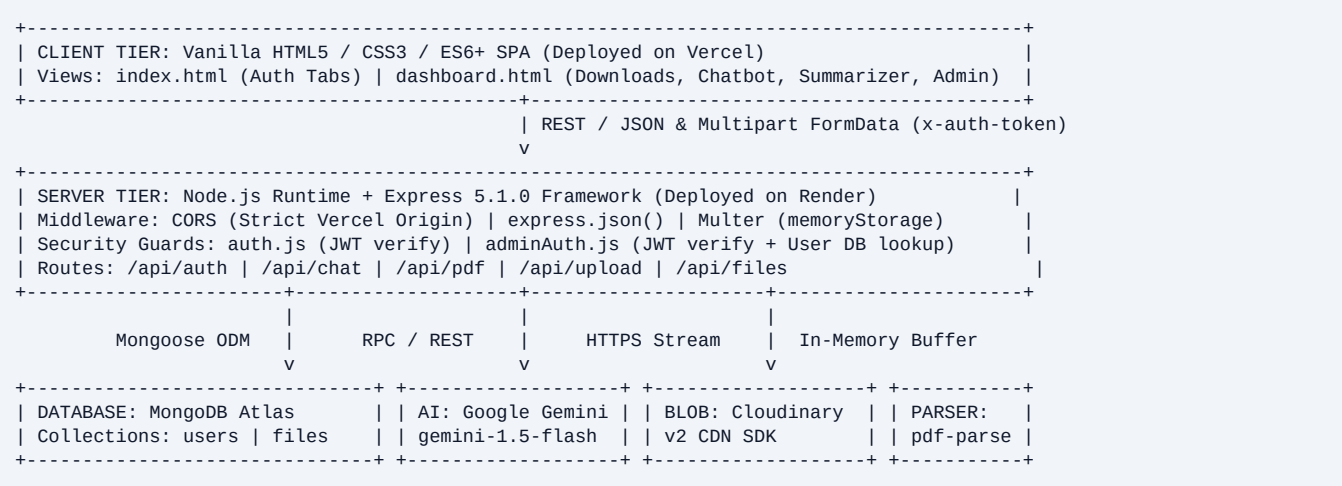
Executive Summary & Scope: This document provides an exhaustive, low-level technical specification of the **DSI Student Portal** codebase. Every architectural decision, authentication handshake, multipart file buffer stream, AI inference lifecycle, MongoDB schema relationship, concurrency limitation, and failure mode has been mapped directly to the actual implemented codebase (no phantom or extraneous dependencies). Designed as an authoritative defense manual, it covers everything required to withstand rigorous, line-by-line technical scrutiny in high-bar SDE interviews.

Table of Contents:

Section 1	End-to-End System Architecture & Detailed Request-Response Lifecycles
Section 2	Core Technical Implementations & Algorithmic Logic (RBAC, In-Memory PDF, Buffer Streaming)
Section 3	Database Design, Mongoose Schemas, Referencing Strategy & Query Patterns
Section 4	Engineering Trade-offs & Justifications (Stack Comparison & Architectural Decisions)
Section 5	Scale Bottlenecks, Failure Modes & Resilience at 10,000 Concurrent Users
Section 6	Edge Cases, Security Vulnerabilities & Defensive Architecture
Section 7	Master SDE Technical Interview Defense Playbook (Top 10 High-Bar Q&A)

1. End-to-End System Architecture & Execution Flow

The DSI Student Portal is engineered as a decoupled, service-oriented web architecture comprising a Vanilla HTML5/CSS3/JavaScript client, an Express 5.1.0 REST API gateway, a MongoDB document store, and external cloud providers for AI generation (Google Gemini 1.5 Flash) and blob storage (Cloudinary v2).



1.1 Complete Request-Response Lifecycles

Lifecycle A: Admin PDF Cloud Upload (`POST /api/upload`)

1. **Client Dispatch:** Admin selects a PDF, inputs title and category, and clicks 'Upload'.
2. **Progress Interception:** Client constructs FormData and dispatches an XMLHttpRequest with xhr.upload.onprogress to compute real-time upload progress: percent = (loaded / total) * 100.
3. **Header Injection:** Attaches x-auth-token: <JWT> to request headers.
4. **CORS Pre-flight & Origin Guard:** Express evaluates cors({ origin: 'https://student-portal-zeta-ashen.vercel.app' }).
5. **Admin RBAC Interception (adminAuth.js):** Extracts x-auth-token. Executes jwt.verify(token, JWT_SECRET). Performs stateful database query: const user = await User.findById(decoded.user.id). Validates if (!user || !user.isAdmin) return res.status(403). Appends req.user = decoded.user and invokes next().
6. **Multer In-Memory Buffering:** multer({ storage: multer.memoryStorage() }).single('pdfFile') captures binary file stream into a Node.js V8 Buffer (req.file.buffer).
7. **Payload Validation:** Verifies req.file exists, and title/category strings are non-empty.
8. **Cloudinary Stream Upload:** Formats buffer into Data URI: data:${req.file.mimetype}; base64,${req.file.buffer.toString('base64')} and executes cloudinary.uploader.upload(...) with a 120-second timeout.
9. **Database Persistence:** Mongoose instantiates new File({ fileName, fileUrl: result.secure_url, uploadedBy: req.user.id, title, category }) and persists it via await newFile.save().
10. **Client UI Synchronization:** Server returns 201 Created. Client resets file inputs, hides progress bar, and invokes fetchAndDisplayFiles() to update the DOM.

Lifecycle B: AI Academic Chat Assistant (`POST /api/chat`)

1. **Client Interaction:** Student inputs question in dashboard.html. DOM appends user bubble and a temporary 'Bot: Thinking...' element.
2. **HTTP POST:** Dispatched via fetch('/api/chat', { method: 'POST', body: JSON.stringify({ message }) }).
3. **Body Parsing:** express.json() parses payload into req.body.message.
4. **Model Instantiation:** genAI.getGenerativeModel({ model: 'gemini-1.5-flash' }) configures the Google Generative AI client.

5. **Remote Inference Execution:** Calls await model.generateContent(userMessage). Google AI executes inference and returns candidate responses.
6. **Extraction & Response:** Controller calls response.text() and returns { response: botResponse }.
7. **DOM Update:** Replaces 'Thinking...' text with formatted AI response and scrolls chat window to bottom.

Lifecycle C: PDF Text Extraction & Extractive Summarization (`POST /api/pdf/summarize`)

1. **Upload:** User submits PDF document via FormData.
2. **Buffer Capture:** Multer buffers raw PDF bytes into req.file.buffer.
3. **Binary Parsing:** pdf-parse parses PDF structure, font streams, and extracts raw text into pdfData.text.
4. **Regex Sentence Tokenization:** Splits raw text using positive lookbehind: extractedText.split(/(?<=[.!?])\s+/).
5. **Salience Selection:** Extracts up to 5 leading sentences or falls back to the first 500 characters.
6. **Response:** Returns { msg, summary } to client.

1.2 Repository Structure & Component Responsibilities

File Path	Layer	Technical Responsibilities & Implemented Logic
`backend/server.js`	Gateway	Initializes Express 5, connects MongoDB via Mongoose, enforces strict CORS origin, mounts all API routes, and serves static frontend.
`backend/middleware/auth.js`	Security	Stateless JWT authentication interceptor. Validates x-auth-token header using HMAC-SHA256 signature verification.
`backend/middleware/adminAuth.js`	Security / RBAC	Two-factor RBAC interceptor. Validates JWT signature AND executes MongoDB query (User.findById) to verify isAdmin === true.
`backend/models/User.js`	Data Model	Mongoose schema for user entities: unique email index, bcrypt hash string, isAdmin boolean, and automatic timestamps.
`backend/models/File.js`	Data Model	Mongoose schema for academic files: Cloudinary secure URL, original filename, title, category, upload date, and foreign-key ref to user.
`backend/routes/auth.js`	Controller	Public endpoints for /register and /login. Implements bcrypt salt/hash (cost 10) and signs 1-hour JWT tokens.
`backend/routes/chat.js`	Controller / AI	Public endpoint POST /api/chat. Connects to Google Generative AI SDK using gemini-1.5-flash for academic Q&A.
`backend/routes/pdf.js`	Controller / NLP	Public endpoint POST /api/pdf/summarize. In-memory buffer extraction via pdf-parse and regex sentence segmentation.
`backend/routes/upload.js`	Controller / Blob	Protected endpoint POST /api/upload (Admin Only). Buffers PDF in memory, converts to Base64 URI, streams to Cloudinary, and saves to MongoDB.
`backend/routes/files.js`	Controller / Query	Protected endpoint GET /api/files (Authenticated). Fetches all uploaded academic resources from MongoDB.
`frontend/index.js`	Client Auth	Manages login/signup tab state, form submissions, token storage in localStorage, and custom modal alert system.
`frontend/dashboard.js`	Client Orchestration	Handles XHR upload with progress bars, multi-predicate download search, AI chat UI stream, PDF summary dispatch, and tab switching.

2. Core Technical Implementations & Algorithmic Logic

2.1 Dual-Tier RBAC State Machine & Security Interceptors

The application enforces role-based access control using two distinct middleware patterns: a **stateless token verifier** (auth.js) and a **stateful database-verified RBAC guard** (adminAuth.js).

```
// 1. Stateless Verification (auth.js) -> O(1) CPU Signature Verification
const decoded = jwt.verify(token, process.env.JWT_SECRET);
req.user = decoded.user;
next();

// 2. Stateful RBAC Verification (adminAuth.js) -> O(1) Signature + O(1) DB Index Lookup
const decoded = jwt.verify(token, process.env.JWT_SECRET);
const user = await User.findById(decoded.user.id); // Primary Key Index lookup
if (!user || !user.isAdmin) {
    return res.status(403).json({ msg: 'Authorization denied: not an admin' });
}
req.user = decoded.user;
next();
```

Architectural Rationale for Live Database Lookup in RBAC:

- **Revocation Defense:** Standard JWTs are stateless and cannot be revoked prior to expiration. If an admin's privileges are demoted, relying solely on token claims (decoded.user.isAdmin) would grant unauthorized access until the 1-hour expiration. Checking User.findById() guarantees real-time privilege enforcement.
- **Deletion Defense:** If an admin user is deleted from MongoDB, the stateful lookup immediately aborts execution with a 403 status, preventing phantom operations.

2.2 In-Memory PDF Parsing & Extractive Summarization Algorithm

Implemented in backend/routes/pdf.js, binary PDF streams are parsed in memory and summarized using regex sentence segmentation:

```
// 1. Binary Buffer Parsing (pdf-parse)
const pdfData = await pdfParse(req.file.buffer);
const extractedText = pdfData.text;

// 2. Lookbehind Sentence Tokenization (Regex Lookbehind)
// Matches whitespace preceded by '.', '!', or '?' without consuming punctuation
const sentences = extractedText.split(/(?<=[.!?])\s+/);

// 3. Salience Window Extraction: Select top-5 leading sentences
const numSentencesForSummary = Math.min(5, sentences.length);
summary = sentences.slice(0, numSentencesForSummary).join(' ') +
          (numSentencesForSummary < sentences.length ? '...' : '');

// 4. Character Boundary Fallback
if (summary.trim().length === 0) {
    summary = extractedText.substring(0, Math.min(extractedText.length, 500)) + '...';
}
```

Complexity Analysis:

- **Time Complexity:** O(L) where L is the character length of extracted text. The regex splits the string in a single linear pass.
- **Space Complexity:** O(L) heap memory allocated for the temporary sentence array.

2.3 Password Hashing via Bcrypt Key Derivation

In backend/routes/auth.js, passwords undergo key derivation via Bcrypt:

```
const salt = await bcrypt.genSalt(10); // Cost factor = 10 (2^10 iterations)
user.password = await bcrypt.hash(user.password, salt);
```

- **Time Complexity:** O(2^R) where R = 10. Takes ~80ms-120ms of CPU compute per registration/login, designed specifically to deter GPU-accelerated brute force attacks.
- **Space Complexity:** O(1) memory overhead (4 KB working state).

2.4 Client-Side Multi-Predicate Search Filter

In frontend/dashboard.js, downloaded academic files are filtered across 4 fields in a single linear pass:

```
files = files.filter(file =>
  file.fileName.toLowerCase().includes(lowerCaseSearchTerm) ||
  (file.title && file.title.toLowerCase().includes(lowerCaseSearchTerm)) ||
  (file.category && file.category.toLowerCase().includes(lowerCaseSearchTerm)) ||
  (file.uploadDate && new Date(file.uploadDate).toLocaleDateString().toLowerCase().includes(lowerCaseSearchTerm))
);
```

- **Time Complexity:** O(M * K) where M is file count and K is average string length.

3. Database Design, Schemas & Query Patterns

3.1 Complete Mongoose Schema Specification

Collection / Field	BSON Type	Index / Constraints	Functional Responsibility & Validation
`users._id`	ObjectId	Primary Key (B-tree)	Default 12-byte BSON identifier (4-byte timestamp + 5-byte random + 3-byte counter).
`users.email`	String	Unique Index	Student/Admin email. Unique index prevents duplicate account registrations.
`users.password`	String	Required	60-character Bcrypt salted hash string (Blowfish-derived format: $2b$10$...).
`users.isAdmin`	Boolean	Default: false	RBAC authorization flag. Governs access to upload and management endpoints.
`users.createdAt / updatedAt`	Date	Automatic	ISO timestamps managed automatically via Mongoose { timestamps: true }.
`files._id`	ObjectId	Primary Key (B-tree)	Unique document identifier for academic files.
`files.fileName`	String	Required	Original file name as extracted from the client's upload payload.
`files.fileUrl`	String	Required	Secure HTTPS CDN URL generated by Cloudinary (e.g. res.cloudinary.com/...).
`files.uploadedBy`	ObjectId	Foreign Key Ref	References _id in the users collection. Tracks uploader ownership.
`files.title`	String	Required	Human-readable academic title (e.g., 'Discrete Math Notes - Module 2').
`files.category`	String	Required	Academic categorization (e.g., 'Notes', 'Question Paper', 'Module').
`files.uploadDate`	Date	Default: Date.now	Document creation timestamp used for sorting and filtering.

3.2 Strategy Justification: Referencing vs. Embedding

The schema implements a **Normalized Document Referencing Strategy** (uploadedBy -> User._id) rather than embedding files inside the User document:

1. **Mitigation of BSON 16MB Document Limit:** In MongoDB, documents cannot exceed 16 megabytes. If an admin uploads thousands of files, embedding file objects inside a user document would lead to document overflow and write failures.
2. **Independent Query Access:** Students frequently query the entire library of files (File.find({})) without needing the user details of the uploader. Embedding would force querying the users collection and slicing arrays.

3.3 Query Patterns & Missing Indices (Interview Critique)

Currently, GET /api/files executes File.find({}) without projection or pagination. Under 10,000 files, this triggers a full collection scan (COLLSCAN). In production, the following compound indices must be added:

```
// Production Index Recommendations:
FileSchema.index({ category: 1, uploadDate: -1 }); // Compound B-Tree Index for category filtering
FileSchema.index({ title: 'text', fileName: 'text' }); // Inverted Full-Text Search Index
FileSchema.index({ uploadedBy: 1 }); // Secondary Index for admin file management queries
```

4. Engineering Trade-offs & Justifications (Interview Defense)

4.1 Technology Stack Comparison Matrix

Component / Layer	Alternative Evaluated	Strategic Justification	Accepted Compromise / Trade-off
Node.js / Express 5	Go (Gin) / Java Spring Boot	Non-blocking asynchronous event loop ideal for I/O-heavy orchestration (MongoDB, Cloudinary, Gemini API). Unified language.	Single-threaded execution vulnerable to event-loop starvation if CPU-heavy parsing (pdf-parse) runs on the main thread.
MongoDB (Mongoose)	PostgreSQL	Flexible schema accommodating evolving academic metadata (tags, courses, departments) without schema migration friction.	Lacks native ACID multi-table joins; integrity checks between File.uploadedBy and User must be enforced at application level.
MemoryStorage Multer	DiskStorage Multer	Essential for ephemeral hosting (Render/Heroku/Vercel) where server containers lack persistent disk storage.	Uploads consume V8 heap memory. Concurrent large uploads risk out-of-memory crashes if unthrottled.
Google Gemini 1.5 Flash	Self-hosted Llama-3 / OpenAI GPT-4o	High tokens-per-second throughput, lower latency, and generous free tier for academic assistance.	Vendor lock-in to Google AI SDK; subject to 15 RPM rate limits on unpaid tiers.
Client Search & Filter	Server-Side Search (Elasticsearch)	Zero backend latency once files are fetched. Instant responsive typing experience for small-to-medium datasets (< 2,000 files).	Network payload grows linearly as file catalog expands; does not scale to tens of thousands of records without pagination.

4.2 Deep Dive into Architectural Decisions

1. Cloudinary Buffer Streaming vs Direct S3 Multipart Upload:
The application buffers files into Node.js memory and base64-encodes them before dispatching to Cloudinary. While this introduces memory pressure, it eliminates the need to configure AWS IAM roles and bucket CORS policies during early prototyping. For enterprise scale, pre-signed upload URLs must be adopted.

2. Extractive Sentence Parsing vs Generative LLM Summarization:
In backend/routes/pdf.js, summarization uses regex lookbehind rather than an LLM call. This architectural decision delivers near-zero latency (< 50ms) and zero API token costs, making it resilient against third-party API rate limits while serving as a rapid baseline preview.

5. Scale Bottlenecks, Failure Modes & Resilience (10k Users)

5.1 Single Points of Failure (SPOF) Under 10k QPS Spikes

1. **V8 Heap Exhaustion (OOM Crash) via Multer memoryStorage:** Storing file buffers in Node.js memory means 100 concurrent 10MB PDF uploads consume 1.0GB of RAM. Base64 serialization (buffer.toString('base64')) inflates payload sizes by an additional 33%, causing immediate Node.js heap exhaustion (FATAL ERROR: CALL_AND_RETRY_LAST Allocation failed - JavaScript heap out of memory).
2. **Event Loop Starvation via pdf-parse:** PDF structure parsing and font stream decompression are synchronous, CPU-intensive operations. When multiple users upload PDFs simultaneously, the V8 event loop freezes, delaying incoming HTTP connections and triggering 504 Gateway Timeouts across all routes.
3. **Cloudinary Socket Exhaustion (120s Timeout):** Cloudinary's timeout is set to 120,000ms. If network congestion occurs, Express holds open socket file descriptors for up to 2 minutes, quickly depleting the operating system's ephemeral port pool.
4. **External Gemini API Rate Limits:** POST /api/chat makes unthrottled calls to Gemini. Under 10,000 requests, Google's rate limiter rejects traffic with 429 Too Many Requests.
5. **Unbounded MongoDB Query (OOM on Client & Server):** File.find({}) retrieves the entire collection into memory without pagination (limit/skip), exhausting network bandwidth and client DOM memory.

5.2 Production Remediation Architecture

Component Layer	Production Technology	Architectural Role & Remediation
File Ingestion	S3 / Cloudinary Pre-signed URLs	Eliminate backend buffer handling. Client uploads binary streams directly to object storage via signed URLs, reducing server RAM usage to zero.
Compute Offload	Node.js Worker Threads / Celery	Offload pdf-parse execution from the main V8 event loop to dedicated worker threads or background worker pools.
Caching Layer	Redis Cluster	Cache /api/files listing in Redis with TTL and cache-aside invalidation on new uploads, reducing database reads by over 90%.
Queue & Throttling	BullMQ + Redis	Queue AI chat requests and PDF summarization jobs with rate limiters to protect external Gemini API quotas.
Database Pagination	Cursor-based Pagination	Replace find({}) with cursor-based pagination (File.find({ _id: { $lt: lastId } }).limit(20)) to guarantee O(1) query complexity.

6. Edge Cases, Security & Defensive Architecture

6.1 Security Vulnerabilities & Defensive Mitigations

Security Vector	Vulnerability in Codebase	Exploit Scenario & Mitigation
Arbitrary File Upload	Multer configuration lacks fileFilter validation.	**Risk:** Attackers can upload executable binaries or HTML files containing malicious scripts. **Fix:** Enforce MIME type checking (application/pdf) and validate magic bytes (%PDF-).
NoSQL Injection	Endpoints pass raw req.body directly into Mongoose queries.	**Risk:** Attackers can pass MongoDB query operators (e.g. {$gt: ''}) to bypass authentication. **Fix:** Sanitize request bodies using express-mongo-sanitize.
JWT Secret Weakness	Signature relies on process.env.JWT_SECRET without algorithm enforcement.	**Risk:** Algorithm confusion attacks if public keys are introduced. **Fix:** Explicitly specify algorithms: ['HS256'] in jwt.verify().
DOM XSS via innerHTML	dashboard.js interpolates file titles and chat responses directly into DOM elements via innerHTML.	**Risk:** Stored XSS if a file title contains script tags. **Fix:** Use textContent or sanitize strings via DOMPurify.
Missing Rate Limiting	Auth, Chat, and Upload routes have no request throttling.	**Risk:** Vulnerable to credential brute-forcing and API denial-of-service. **Fix:** Implement express-rate-limit backed by Redis.

6.2 Concrete Application Edge Cases

1. **Scanned Image PDFs (Zero Text Extraction):** pdf-parse extracts embedded text streams. If a student uploads a scanned image PDF, extracted text is empty (extractedText.length === 0). The code handles this via a fallback message: *'The PDF file contains no readable text or text extraction failed.'* (Production fix: Integrate Tesseract OCR).
2. **Bcrypt 72-Byte Truncation Limit:** Bcrypt silently truncates passwords beyond 72 bytes. Passwords longer than 72 bytes can be manipulated without altering the hash. Server-side validation should enforce a 72-byte upper bound.
3. **CORS Configuration Lock:** server.js restricts CORS to https://student-portal-zeta-ashen.vercel.app. Local frontend testing on localhost:3000 or 127.0.0.1:5500 will fail with CORS origin rejection unless an environment-based CORS list is used.
4. **Stale Admin Privilege Invalidation:** adminAuth.js prevents stale privilege escalation by performing a live MongoDB lookup (User.findById), ensuring revoked admins cannot access administrative routes.

7. Master SDE Interview Defense Playbook (Top 10 High-Bar Q&A)

Q1: Why does adminAuth.js make an asynchronous database call while auth.js does not?

auth.js relies on stateless JWT signature verification (O(1) CPU) for standard student routes to minimize database load. In contrast, adminAuth.js guards high-privilege mutation routes (such as file uploads). If an admin is demoted or deleted, relying solely on token claims would allow unauthorized access until token expiration. Making a live database call via User.findById ensures immediate revocation defense.

Q2: What is the failure mode of using multer.memoryStorage() under high concurrency?

Multer buffers uploaded files directly into V8 heap memory. Under high traffic, concurrent multi-megabyte PDF uploads quickly consume available RAM. Furthermore, encoding buffers to Base64 increases memory usage by 33%. When heap consumption exceeds the default Node.js limit (~1.4GB), the process crashes with an Out-of-Memory (OOM) error. Production systems should stream directly to S3 via pre-signed URLs.

Q3: How does your PDF summarizer handle multi-column academic papers or scanned documents?

The current implementation uses pdf-parse to extract raw text streams, followed by regex sentence segmentation (/(?<=[.!?])\s+/). While fast and low-cost, this approach has limitations: it flattens multi-column text in reading order and cannot extract text from scanned image PDFs. In production, this can be improved by integrating OCR (e.g., Tesseract) and using layout-aware NLP models.

Q4: Why did you choose document referencing over embedding for the User-File relationship?

In MongoDB, documents cannot exceed 16 megabytes. A one-to-many relationship where an admin uploads thousands of files would eventually exceed this document size limit if files were embedded within the User document. Referencing ensures file metadata is stored independently while allowing efficient global queries via File.find().

Q5: What happens if the Gemini API experiences latency or rate limiting during a chat request?

Because the route awaits model.generateContent() directly within the request handler, slow responses from Google AI hold open the client HTTP connection. If traffic spikes, rate limits trigger 429 errors. To resolve this, AI tasks should be processed through a background job queue (e.g., BullMQ) or streamed to the client via Server-Sent Events (SSE).

Q6: How do you prevent NoSQL injection in your MongoDB queries?

In routes like /api/auth/login, queries accept inputs directly (User.findOne({ email })). If an attacker passes an object like { email: { $gt: '' } }, the query could match unintended records. To protect against this, inputs should be sanitized using express-mongo-sanitize and validated against strict schemas using Zod or Joi.

Q7: Why is bcrypt.genSalt(10) considered appropriate for password hashing?

A cost factor of 10 represents 2^10 (1,024) hashing rounds, requiring roughly 80–120ms of CPU compute per password. This introduces sufficient delay to thwart GPU-based brute-force attacks while remaining fast enough to prevent user-perceivable login latency.

Q8: What performance issues arise from using File.find({}) without pagination?

Querying all files without pagination or projection performs a full collection scan (COLLSCAN) and transfers the entire dataset over the network on every request. As the library grows, this increases memory usage on both the server and client. Adding cursor-based pagination (limit/skip) ensures constant-time O(1) query complexity.

Q9: How do you defend against Stored XSS when rendering file titles and categories in dashboard.js?

dashboard.js currently uses innerHTML to render file listings. If an admin inputs a malicious script tag in a file title, that script executes in the user's browser session. To eliminate this risk, titles should be inserted using textContent, or sanitized through libraries like DOMPurify.

Q10: Why did you implement strict CORS origin matching rather than using wildcard access?

server.js explicitly restricts access to 'https://student-portal-zeta-ashen.vercel.app'. Using a wildcard (*) allows third-party malicious websites to make authenticated cross-origin requests on behalf of users if tokens are exposed. Strict origin matching ensures only authorized client applications can communicate with the API.